

Image2Card: 基于视觉语言模型的图片知识卡片管理系统

《程序设计实习》大作业报告 · 赛道：生活工具

组号 114

队名 超躺小登队

成员 张煦恒 2300019813 · 夏海闻 2300010813 日期 2026 年 7 月

代码仓库 github.com/catburgg/Image2Card_submission (Python 3.12 · PyQt5)

一、程序功能介绍

1.1 项目定位与要解决的痛点

我们每天都会遇到大量“图片形态的知识”——餐厅菜单上的陌生菜名、景区路牌上的地名典故、学习资料上的专业名词。这些信息往往一看而过、难以沉淀。传统 OCR 工具只能把图片“抄成文字”，却无法理解“这是什么、值不值得记、和我已知的知识有什么关系”。

Image2Card 是一个带图形界面的桌面应用：用户导入一张图片，系统借助视觉语言模型 (VLM) 自动识别其中值得学习的词条，为每个词条生成一张结构化的知识卡片（含摘要、详解、外链、关联词），并沉淀进个人知识库中，支持可交互标注、知识图谱、间隔重复复习、卡片对话等一系列学习闭环。它把“看到图片”转化为“可管理、可复习、可关联的个人知识”。

1.2 核心功能

功能模块	说明
图片导入与管理	支持导入本地图片、按工作区（分类）组织；每张来源图片可追溯其产出的卡片。
VLM 知识抽取	OCR (PaddleOCR) + 视觉语言模型联合分析，输出带 bbox 的结构化 JSON：场景类型、词条、候选卡片；对模型返回的非法 JSON 自动要求纠正。
知识状态参与识别	把用户已有卡片与“已拒绝/已熟悉”历史注入 Prompt，引导模型不重复推荐、聚焦真正陌生的词条。
人机协同确认	模型结果不直接入库，而是进入候选队列，用户可新建 / 合并 / 忽略 / 编辑后再确认，保证知识库质量。
卡片库	每张卡片区分模型笔记与用户笔记；支持熟悉度、关联词、外链、分类工作区、全文搜索与联想补全。
图片标注视图	在原图上叠加可交互 bbox：悬停显示缩略卡片、点击展开详情、支持缩放与平移。
知识图谱	将卡片与关联关系渲染成聚团式知识地图，节点大小由 PageRank 重要度与熟悉度决定，支持社区聚团。
间隔重复复习	基于 FSRS 算法排期，以“百词斩式”匿名选择题组织复习会话，可跳过已熟悉卡片。
卡片对话	针对单张卡片与大模型多轮对话，深入追问词条相关知识。
可自定义识别器	识别层抽象为统一接口，支持 openai / qwen / gemini / claude 等多家 Provider 与 mock 离线模式。

1.3 典型使用流程

导入图片 → VLM 分析得到候选词条（图上高亮 bbox） → 用户在候选队列中确认/编辑 → 卡片入库并生成关联 → 在卡片库/知识图谱中浏览 → 发起复习会话或与卡片对话，完成学习闭环。

1.4 界面展示

应用采用统一的暗色主题，含首页、导入识别、卡片库/知识图谱、图库、复习、卡片对话、设置等多个页面与对话框。以下为实际运行截图（示例数据：13 张卡片、3 个主题聚团）。



图 1 知识图谱：卡片按主题聚团，节点大小体现重要度/熟悉度，右侧为卡片详情面板



图 2 卡片对话：勾选卡片后与大模型多轮问答，左列显示模型生成的摘要与熟悉度

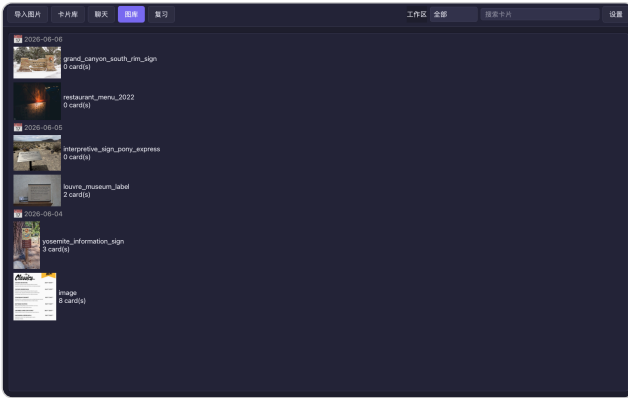


图 3 图库：来源图片按日期归档，并显示其产生的卡片数量

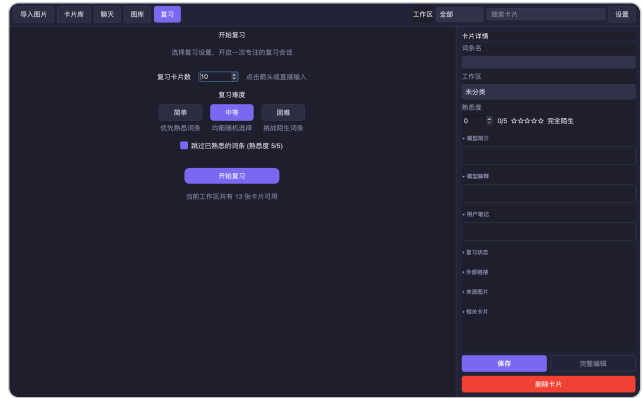


图 4 复习：基于 FSRS 排期的复习会话设置，支持跳过已熟悉词条

二、项目各模块与类设计细节

2.1 总体架构：四层解耦

项目采用清晰的分层架构，自底向上依次为数据模型层、持久化层、核心服务层、界面表现层；上层只依赖下层的抽象接口，AppContext 单例负责组装并持有全部后端服务，实现 UI 与业务逻辑的彻底解耦。

界面表现层 **ui/** MainWindow · AppContext(单例) · AnalyzePage / LibraryPage / CardMapPage / ReviewSessionPage / CardChatPage / SettingsPage · 各类 Widget

核心服务层 **core/** ImageParser · OCRService · ImageAnalysisService · CandidateManager · CardManager · GraphEngine · ReviewEngine · ReviewSession · CardChatService · SearchEngine · UserPreferences · ImageRenderer

持久化层 **storage/** Repository (KnowledgeBase 内存对象图 $\rightleftharpoons$ SQLite, 路径可移植)

数据模型层 **dataclass/models.py** Card · KnowledgeBase · Category · SourceImage · AnalyzeResult · 枚举 · pydantic 校验模型

2.2 数据模型层 dataclass/models.py

全部核心数据结构的单一事实来源，用 @dataclass 与 pydantic.BaseModel 定义：

- 枚举: CandidateStatus (候选状态: 新建/已存在/已拒绝…)、RelationType (关系类型)、ReviewRating (复习评分)。

- 领域对象: Card (知识卡片, 含模型笔记/用户笔记分离、熟悉度、FSRS 排期字段、关联词、外链、bbox)、Category (分类工作区)、SourceImage (来源图片)、BoundingBox、Link。
- 识别契约 (pydantic): VLMBBox / VLMLink / VLMResultItem / VLMCandidateCard / AnalyzeResult, 用于严格校验视觉语言模型返回的 JSON。
- 聚合根: KnowledgeBase 作为内存对象图统一持有全部卡片、分类与关系; UserConfig 保存全局配置。

2.3 持久化层 storage/repository.py

Repository 类将内存中的 KnowledgeBase 同步到 SQLite，并在启动时从数据库重建内存对象图。特别实现了路径可移植逻辑——将 ~/.image2card 下的绝对图片路径与相对路径互相转换，使数据目录可在不同机器/账户间迁移（对应提交记录“make image2card data portable”）。

2.4 核心服务层 core/

类 / 模块	职责
ImageParser (vlm.py)	把“图片 + 配置 + 已有卡片上下文”发送给 VLM，校验返回 JSON，格式出错时自动要求模型修正，产出 AnalyzeResult。VLMConfig/VLMInput 描述调用参数与多 Provider 配置。
OCRService (ocr_service.py)	封装 PaddleOCR 初始化与调用，支持后台预加载模型以消除冷启动延迟。
ImageAnalysisService (image_manager.py)	编排“图片上传 → VLM 分析 → 与 KB 比对 → 生成候选列表”的完整流程。
CandidateManager (candidate.py)	接收 AnalyzeResult，与 KnowledgeBase 比对生成带状态的 PendingCandidate 列表，驱动人机确认流程。
CardManager (card_manager.py)	在 KB（内存）+ Repository（SQLite）之上提供统一的卡片 CRUD、关系管理与分类树管理。
GraphEngine (graph.py)	将卡片转化为知识地图的 GraphNode/GraphEdge/GraphCluster/GraphData，计算布局、社区聚团与 PageRank 重要度。
ReviewEngine (review.py)	FSRS 间隔重复引擎，依据卡片的 stability / difficulty / next_due 等字段计算下次复习时间。
ReviewSession (review_session.py)	组织一次“百词斩式”复习会话：词条匿名化、生成选择题、记录评分。
CardChatService (card_chat.py)	围绕单张卡片与大模型进行多轮对话。
SearchEngine (search.py)	KB 之上的多字段加权全文搜索与联想补全。
UserPreferences (user_preferences.py)	记录用户对候选词条的拒绝/熟悉决策，反哺 VLM Prompt。
ImageRenderer (renderer.py)	可交互标注渲染器（QGraphicsView 体系），实现 bbox 悬停缩略卡、点击详情、缩放平移。

2.5 界面表现层 ui/

采用 PyQt5，以 MainWindow 为主窗口、侧边导航切换多个功能页面（AnalyzePage 图片识别、LibraryPage 卡片库、CardMapPage 知识图谱、ReviewSessionPage 复习、CardChatPage 对话、SettingsPage 设置），并配合卡片详情对话框、候选标注视图、聚团图谱控件等组件，满足“多窗口/多对话框”的要求。AppContext 以单例形式注入全部后端服务，页面之间通过它共享状态。

面向对象与工程亮点：① 严格分层、依赖倒置，UI 不含业务逻辑；② 识别层抽象为统一 Analyzer 接口，支持多 Provider 与 mock 离线运行；③ pydantic 对模型输出做契约式校验并自动纠错；④ 内存对象图 + SQLite 双写，兼顾交互性能与持久化；⑤ 全项目使用 logging 记录关键流程，便于调试。

三、小组成员分工情况

本项目为 2 人协作，基于 GitHub 进行代码组织与协同开发，两位成员均全程参与需求讨论、代码评审与联调测试，具体分工如下。

成员	学号	主要负责	工作量
夏海闻	2300010813	整体框架设计、OCR 与复习模块：项目分层架构与数据/存储层（ <code>app.py</code> / <code>app_context.py</code> / <code>models.py</code> / <code>repository.py</code> ）、OCR 文字识别（ <code>ocr_service.py</code> ）、FSRS 间隔重复复习引擎与复习会话（ <code>review.py</code> / <code>review_session.py</code> / <code>review_session_page.py</code> ）	约 1/2
张煦恒	2300019813	知识图谱与图卡关系构建：知识图谱生成与可视化（ <code>graph.py</code> / <code>card_graph_widget.py</code> ）、图片理解与“图片 → 卡片”关系构建（ <code>vlm.py</code> / <code>image_manager.py</code> / <code>candidate.py</code> / <code>card_manager.py</code> / <code>renderer.py</code> ）	约 1/2

四、项目总结与反思

4.1 完成情况

我们完整实现了“图片 → 结构化知识卡片 → 沉淀 → 复习/关联”的学习闭环，涵盖图片管理、VLM 知识抽取、人机协同确认、卡片库、可交互标注、知识图谱、FSRS 复习、卡片对话、全文搜索等模块，界面包含多个窗口与对话框，代码以面向对象方式分层组织，核心逻辑经 mock 模式与真实 API 双路径联调验证。

4.2 技术亮点

- VLM 驱动的图片知识抽取：以“图片理解”替代传统 OCR + 关键词，输出场景感知的结构化结果。
- 用户知识状态参与识别：已有卡片与拒绝历史反哺 Prompt，让识别“越用越懂你”。
- 人机协同的确认流程：候选队列而非直接入库，兼顾自动化与知识库质量。
- 可视化强：原图 bbox 交互标注 + PageRank 聚团知识图谱，直观体现知识关联。

4.3 遇到的挑战与解决

- VLM 给不出精确 bbox，必须借助 OCR 精确工具调用：这是本项目最关键的挑战。视觉语言模型擅长语义理解，但直接让它输出文字区域的坐标框往往明显偏移、不可靠，无法直接用于原图标注。我们据此采用“语义与定位解耦”策略——VLM 只负责识别值得学习的词条及其含义，再把 PaddleOCR 作为精确定位工具调用，以 OCR 检测到的文本框为词条匹配 / 校正 bbox，才能在原图上得到准确的可交互标注框。
- VLM 返回 JSON 不稳定：用 pydantic 定义严格契约，解析失败时自动回传错误并要求模型修正，显著提升稳定性。
- OCR 冷启动慢：应用启动后用后台线程预加载 PaddleOCR 模型，避免首次分析卡顿。
- 坐标映射：图片缩放/平移下 bbox 与原图坐标的换算，通过 QGraphicsView 场景坐标系统一处理。
- 数据可移植：重构图片路径存储为相对 `~/image2card` 的可迁移形式，解决跨机器复制后失效的问题。

4.4 不足与改进方向

当前系统最大的不足是端到端时延偏长：一次图片分析需串行经过 OCR 检测与 VLM 推理两个较重的阶段，且 VLM 为一次性阻塞式调用，用户要等整轮完成才能看到结果，交互体感不够顺滑。未来最主要的改进方向是引入流式（streaming）与异步框架——让 VLM 结果边生成边返回、OCR 与 VLM 阶段并行 / 流水化处理，并用异步任务队列避免 UI 阻塞，从而显著降低用户可感知时延。其次，识别质量依赖外部 VLM API、离线能力有限，可引入本地小模型兜底；知识图谱在卡片规模很大时的布局性能、以及 FSRS 参数个性化，也仍有进一步优化空间。

4.5 收获

通过本项目，我们完整实践了一个中等规模 Python 桌面应用的面向对象设计——分层架构、接口抽象、数据契约、持久化同步与 GUI 工程化，并在团队协作、Git 流程与联调测试中积累了宝贵经验。